Lab 5: Static Code Analysis

Digital Wallet and Payment Tracker

SRN: PES1UG24CS087
Primary script: digital_wallet.py
Assigned repository: L5_29_digital_wallet
Tools used: Pylint, Bandit, Flake8

Tool	Before	After
Pylint	5.69/10	10.00/10
Bandit	4 issues (1 High, 1 Medium, 2 Low)	0 issues
Flake8	4 PEP 8 violations	0 violations

This consolidated report documents the static analysis of the assigned digital wallet program, the issues found by the three tools, the fixes applied, the before-and-after tool output, and a reflection on the exercise.

1. Introduction to Static Code Analysis

Static code analysis is the process of examining source code without executing it. It helps developers find potential bugs, security weaknesses, maintainability problems, and style violations early, before the program reaches testing or deployment. Because the analysis is automated, it can be applied consistently to every change and can catch issues that are easy for a human reviewer to miss.

This lab used three complementary tools. Pylint reviews code quality and logic-related problems such as poor naming, missing documentation, mutable default arguments, unused arguments, and global state. Bandit focuses on security by detecting risky APIs and insecure coding patterns. Flake8 enforces PEP 8 style rules covering whitespace, line length, spacing around operators, and other formatting concerns.

The three tools in this lab

- **Pylint:** A strict code reviewer that reports code quality, design, naming, documentation, and logical issues, and assigns a score out of 10.
- **Bandit:** A security-focused analyzer that flags insecure functions and patterns such as weak hashing, insecure temporary files, hardcoded secrets, and non-cryptographic randomness.
- **Flake8:** A PEP 8 style checker that combines formatting and style checks to keep the code readable and consistent.

The target program simulates a small digital wallet. It generates an OTP, adds money, applies a daily limit, checks a PIN before merchant payments, prints a mini statement, calculates cashback, and verifies a gateway secret. The original script ran successfully but contained several quality, security, and formatting defects.

2. Issues Identified Before Fixing

The screenshots below show the actual tool output captured before the code was changed. The reports were saved as text files and then rendered as terminal screenshots for this consolidated PDF.

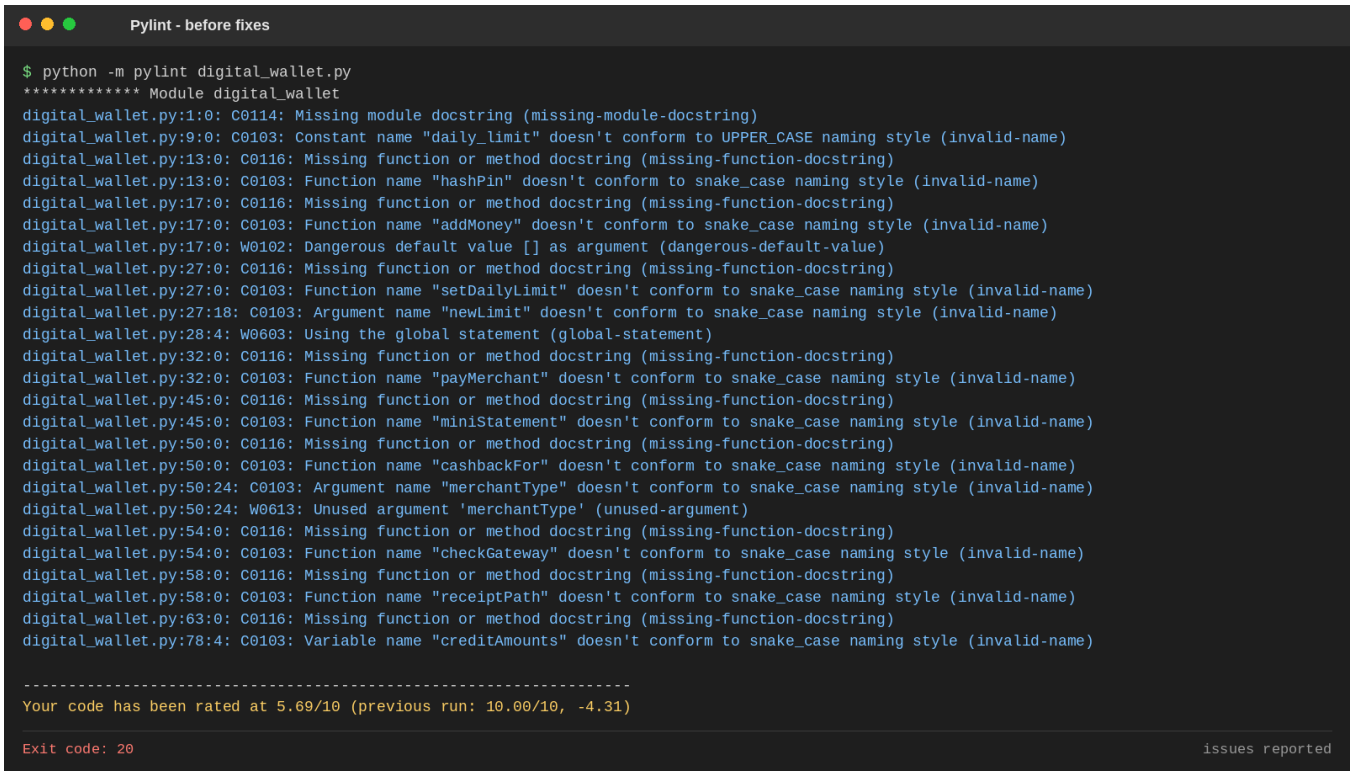


Figure 1. Pylint findings before fixes: score 5.69/10 with naming, documentation, mutable-default, global, and unused-argument warnings.

Bandit - before fixes

```
$ python -m bandit -r digital_wallet.py
Run started:2026-10-28 15:58:48.731651+00:00

Test results:
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'sk_demo_wallet_0000'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html
   Location: ./digital_wallet.py:6:17
5▫
6▫GATEWAY_SECRET = "sk_demo_wallet_0000"
7▫WALLET_PIN = "4321"

--------------------------------------------------
>> Issue: [B324:hashlib] Use of weak SHA1 hash for security. Consider usedforsecurity=False
   Severity: High   Confidence: High
   CWE: CWE-327 (https://cwe.mitre.org/data/definitions/327.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b324_hashlib.html
   Location: ./digital_wallet.py:14:11
13▫def hashPin(pin):
14▫    return hashlib.sha1(pin.encode()).hexdigest()
15▫

--------------------------------------------------
>> Issue: [B306:blacklist] Use of insecure and deprecated function (mktemp).
   Severity: Medium   Confidence: High
   CWE: CWE-377 (https://cwe.mitre.org/data/definitions/377.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b306-mktemp-q
   Location: ./digital_wallet.py:60:11
59▫    # Helper kept for the lab: not called by the demo
60▫    return tempfile.mktemp(suffix=".pdf")
61▫

--------------------------------------------------
>> Issue: [B311:blacklist] Standard pseudo-random generators are not suitable for security/cryptographic purposes.
   Severity: Low   Confidence: High
   CWE: CWE-330 (https://cwe.mitre.org/data/definitions/330.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b311-random
   Location: ./digital_wallet.py:64:10
63▫def main():
64▫    otp = random.randint(100000, 999999)
65▫    print("OTP sent:", otp)

--------------------------------------------------

Code scanned:
▫Total lines of code: 60
▫Total lines skipped (#nosec): 0
▫Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
▫Total issues (by severity):
▫▫Undefined: 0
▫▫Low: 2
▫▫Medium: 1
▫▫High: 1
▫Total issues (by confidence):
▫▫Undefined: 0
▫▫Low: 0
▫▫Medium: 1
▫▫High: 3
Files skipped (0):

Exit code: 1                                                    issues reported
```

Figure 2. Bandit findings before fixes: weak SHA-1 hashing (High), insecure mktemp (Medium), hardcoded secret (Low), and insecure randomness (Low).

Flake8 - before fixes

```
$ python -m flake8 digital_wallet.py
digital_wallet.py:76:51: E261 at least two spaces before inline comment
digital_wallet.py:78:18: E225 missing whitespace around operator
digital_wallet.py:79:28: E231 missing whitespace after ','
digital_wallet.py:80:1: E305 expected 2 blank lines after class or function definition, found 0

Exit code: 1                                                    issues reported
```

Figure 3. Flake8 findings before fixes: four PEP 8 formatting violations.

3. Issue Identification Table

The table documents the identified issues in severity order, with high and medium findings first. More than the required minimum of four issues are included.

Issue	Type	Line(s)	Description	Fix Approach
Weak password hashing	Bandit B324 (High)	14	The PIN was hashed with SHA-1, which is not suitable for security-sensitive hashing.	Replaced SHA-1 with SHA-256 in hash_pin.
Insecure temporary file creation	Bandit B306 (Medium)	60	tempfile.mktemp creates a predictable filename and is vulnerable to race conditions.	Replaced mktemp with tempfile.mkstemp and closed the returned descriptor immediately.
Hardcoded gateway secret	Bandit B105 (Low)	6	The gateway secret was embedded directly in the source code.	Read the secret from the GATEWAY_SECRET environment variable instead.
Insecure OTP generation	Bandit B311 (Low)	64	The OTP used random, which is not cryptographically secure.	Replaced random.randint with secrets.randbelow.
Mutable default argument	Pylint W0102 (Bug)	17	The default log=[] list was shared across calls, so data could leak between invocations.	Changed the default to None and initialized a new list inside the function.
Global statement	Pylint W0603	28	set_daily_limit modified daily_limit using a global statement.	Stored the daily limit in the wallet dictionary and removed the global variable.
Non-standard naming	Pylint C0103	9, 13, 17, 27, 32, 45, 50, 54, 58, 78	Function, argument, and variable names used camelCase or lowercase constant naming.	Renamed identifiers to snake_case and used UPPER_CASE where appropriate.
Missing documentation	Pylint C0114/C0116	1, 13-63	The module and its functions lacked docstrings.	Added a module docstring and concise docstrings for each function.
Unused argument	Pylint W0613	50	The merchantType argument in cashbackFor was never used.	Renamed it to _merchant_type to mark it intentionally unused.
PEP 8 formatting violations	Flake8 E261/E225/E231/E305	76, 78, 79, 80	Inline comments and operators were spaced incorrectly, a comma lacked a following space, and two blank lines were missing before the top-level call.	Corrected spacing, added the missing blank lines, and wrapped an overlong line.

4. Before-and-After Tool Output

This section contains the complete contents of all six text reports: Pylint, Bandit, and Flake8 before and after the fixes. An empty Flake8 after-report is shown explicitly.

4.1 Pylint Before

```
************* Module digital_wallet
digital_wallet.py:1:0: C0114: Missing module docstring (missing-module-docstring)
digital_wallet.py:9:0: C0103: Constant name "daily_limit" doesn't conform to UPPER_CASE naming style (invalid-name)
digital_wallet.py:13:0: C0116: Missing function or method docstring (missing-function-docstring)
digital_wallet.py:13:0: C0103: Function name "hashPin" doesn't conform to snake_case naming style (invalid-name)
digital_wallet.py:17:0: C0116: Missing function or method docstring (missing-function-docstring)
digital_wallet.py:17:0: C0103: Function name "addMoney" doesn't conform to snake_case naming style (invalid-name)
digital_wallet.py:17:0: W0102: Dangerous default value [] as argument (dangerous-default-value)
digital_wallet.py:27:0: C0116: Missing function or method docstring (missing-function-docstring)
digital_wallet.py:27:0: C0103: Function name "setDailyLimit" doesn't conform to snake_case naming style (invalid-name)
digital_wallet.py:27:18: C0103: Argument name "newLimit" doesn't conform to snake_case naming style (invalid-name)
digital_wallet.py:28:4: W0603: Using the global statement (global-statement)
digital_wallet.py:32:0: C0116: Missing function or method docstring (missing-function-docstring)
digital_wallet.py:32:0: C0103: Function name "payMerchant" doesn't conform to snake_case naming style (invalid-name)
digital_wallet.py:45:0: C0116: Missing function or method docstring (missing-function-docstring)
digital_wallet.py:45:0: C0103: Function name "miniStatement" doesn't conform to snake_case naming style (invalid-name)
digital_wallet.py:50:0: C0116: Missing function or method docstring (missing-function-docstring)
digital_wallet.py:50:0: C0103: Function name "cashbackFor" doesn't conform to snake_case naming style (invalid-name)
digital_wallet.py:50:24: C0103: Argument name "merchantType" doesn't conform to snake_case naming style (invalid-name)
digital_wallet.py:50:24: W0613: Unused argument 'merchantType' (unused-argument)
digital_wallet.py:54:0: C0116: Missing function or method docstring (missing-function-docstring)
digital_wallet.py:54:0: C0103: Function name "checkGateway" doesn't conform to snake_case naming style (invalid-name)
digital_wallet.py:58:0: C0116: Missing function or method docstring (missing-function-docstring)
digital_wallet.py:58:0: C0103: Function name "receiptPath" doesn't conform to snake_case naming style (invalid-name)
digital_wallet.py:63:0: C0116: Missing function or method docstring (missing-function-docstring)
digital_wallet.py:78:4: C0103: Variable name "creditAmounts" doesn't conform to snake_case naming style (invalid-name)

-------------------------------------------------------------------
Your code has been rated at 5.69/10 (previous run: 10.00/10, -4.31)
```

4.2 Pylint After

```
-------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

4.3 Bandit Before

```
Run started:2026-10-28 15:58:48.731651+00:00

Test results:
&gt;&gt; Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'sk_demo_wallet_0000'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html
   Location: ./digital_wallet.py:6:17
5▫
6▫GATEWAY_SECRET = "sk_demo_wallet_0000"
7▫WALLET_PIN = "4321"

--------------------------------------------------
&gt;&gt; Issue: [B324:hashlib] Use of weak SHA1 hash for security. Consider usedforsecurity=False
   Severity: High   Confidence: High
   CWE: CWE-327 (https://cwe.mitre.org/data/definitions/327.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b324_hashlib.html
   Location: ./digital_wallet.py:14:11
13▫def hashPin(pin):
14▫    return hashlib.sha1(pin.encode()).hexdigest()
15▫

--------------------------------------------------
&gt;&gt; Issue: [B306:blacklist] Use of insecure and deprecated function (mktemp).
   Severity: Medium   Confidence: High
   CWE: CWE-377 (https://cwe.mitre.org/data/definitions/377.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b306-mktemp-q
   Location: ./digital_wallet.py:60:11
59▫    # Helper kept for the lab: not called by the demo
60▫    return tempfile.mktemp(suffix=".pdf")
61▫

--------------------------------------------------
&gt;&gt; Issue: [B311:blacklist] Standard pseudo-random generators are not suitable for security/cryptographic purposes.
   Severity: Low   Confidence: High
   CWE: CWE-330 (https://cwe.mitre.org/data/definitions/330.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b311-random
   Location: ./digital_wallet.py:64:10
63▫def main():
64▫    otp = random.randint(100000, 999999)
65▫    print("OTP sent:", otp)
```

```
--------------------------------------------------

Code scanned:
	Total lines of code: 60
	Total lines skipped (#nosec): 0
	Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
	Total issues (by severity):
		Undefined: 0
		Low: 2
		Medium: 1
		High: 1
	Total issues (by confidence):
		Undefined: 0
		Low: 0
		Medium: 1
		High: 3
Files skipped (0):
```

4.4 Bandit After

```
Run started:2026-10-28 16:00:03.091456+00:00

Test results:
	No issues identified.

Code scanned:
	Total lines of code: 75
	Total lines skipped (#nosec): 0
	Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
	Total issues (by severity):
		Undefined: 0
		Low: 0
		Medium: 0
		High: 0
	Total issues (by confidence):
		Undefined: 0
		Low: 0
		Medium: 0
		High: 0
Files skipped (0):
```

4.5 Flake8 Before

```
digital_wallet.py:76:51: E261 at least two spaces before inline comment
digital_wallet.py:78:18: E225 missing whitespace around operator
digital_wallet.py:79:28: E231 missing whitespace after ','
digital_wallet.py:80:1: E305 expected 2 blank lines after class or function definition, found 0
```

4.6 Flake8 After

```
(empty file - Flake8 reported no violations)
```

5. Results After Fixing the Issues

After applying the fixes, the script still produced the same functional output. Pylint reached a perfect score, Bandit reported no security issues, and Flake8 produced no formatting violations.

Pylint - after fixes

```
$ python -m pylint digital_wallet.py

--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)

Exit code: 0                                                clean run
```

Figure 4. Pylint after fixes: score 10.00/10 and no warnings.

Bandit - after fixes

```
$ python -m bandit -r digital_wallet.py
Run started:2026-10-28 16:00:03.091456+00:00

Test results:
▫No issues identified.

Code scanned:
▫Total lines of code: 75
▫Total lines skipped (#nosec): 0
▫Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
▫Total issues (by severity):
▫▫Undefined: 0
▫▫Low: 0
▫▫Medium: 0
▫▫High: 0
▫Total issues (by confidence):
▫▫Undefined: 0
▫▫Low: 0
▫▫Medium: 0
▫▫High: 0
Files skipped (0):

Exit code: 0                                                clean run
```

Figure 5. Bandit after fixes: no issues identified.

Flake8 - after fixes

```
$ python -m flake8 digital_wallet.py
(no output - the tool reported no issues)

Exit code: 0                                                clean run
```

Figure 6. Flake8 after fixes: clean run with zero violations.

6. Reflection Questions and Answers

1. What is the purpose and scope of static code analysis, and what did each tool contribute?

Static analysis examines source code without running it. Its scope includes code quality, security, maintainability, style, and potential logic defects. Pylint contributed quality and design feedback: it identified missing docstrings, non-standard names, a mutable default argument, a global statement, and an unused argument. Bandit contributed security feedback: it found weak SHA-1 hashing, insecure temporary-file creation, a hardcoded gateway secret, and the use of non-cryptographic randomness for an OTP. Flake8 contributed consistency by finding PEP 8 formatting issues such as incorrect spacing and missing blank lines.

2. Which common Python issues did you identify and resolve, and how did severity affect your priorities?

The highest-priority finding was Bandit B324, the use of SHA-1 for PIN hashing. This was fixed with SHA-256. The medium-severity finding was the use of tempfile.mktemp, which was replaced with tempfile.mkstemp and an immediate close of the descriptor. The low-severity Bandit findings were the hardcoded gateway secret and the use of random for OTP generation; these were fixed with an environment variable and secrets.randbelow. Pylint's mutable default argument was treated as a real bug because the shared list could leak state between calls. Naming, documentation, global state, unused arguments, and Flake8 formatting were addressed after the security and bug findings.

3. How did you interpret the reports and ensure the fixes did not change program behavior?

Each report was read from the highest-severity line downward. Bandit's severity labels made the security order clear, while Pylint's message codes identified the exact quality rule being broken. Flake8's file-and-line output made the style fixes straightforward. After each batch of changes, the program was executed again and the output was compared with the original run. The displayed transactions, declined payments, cashback, gateway check, final balance, and total credited amount remained the same. The tool reports were then regenerated to confirm that the issues had disappeared and that no new issues had been introduced.

4. How does static analysis contribute to long-term code quality and maintainability, and what limitations did you observe?

Static analysis makes quality and security checks repeatable and consistent. It catches risky APIs, shared-state bugs, naming problems, and formatting drift before code review or testing. This reduces review effort and makes the codebase easier to maintain. The main limitation is that static tools do not understand the program's intended behavior: a clean report does not prove that the logic is correct. Some warnings are also stylistic or context-dependent, so findings must be reviewed rather than fixed blindly. In future work, I would run these tools regularly during development and in CI, use the severity ordering to address security first, and combine static analysis with tests and manual review.

7. Conclusion

The digital wallet program was successfully cleaned without changing its observable behavior. Pylint improved from 5.69/10 to 10.00/10, Bandit went from four security findings to zero, and Flake8 went from four formatting violations to zero. The exercise showed how the three tools cover different concerns and how their reports can be used together to improve security, correctness, readability, and maintainability.